Data Science sous R

Chapitre 2 — Introduction au langage R

ESME — 2026-2027

Table des matières

2 Introduction au langage R **2**
2.1 R : un peu d'histoire . . . 2
2.2 Introduction à RStudio . . . 2
2.3 Premiers pas : R comme calculatrice . . . 5
2.4 Variables et assignation . . . 6
2.5 Types de données . . . 7
2.6 Structures de données . . . 9
2.7 Opérateurs . . . 12
2.8 Accéder aux éléments et les modifier . . . 14
2.9 Structures algorithmiques . . . 16
2.10 Fonctions et programmation modulaire . . . 18
2.11 Packages et aide . . . 20
2.12 Résumé . . . 20
2.13 Exercices . . . 21

Chapitre 2

Introduction au langage R

Objectifs du chapitre

- Prendre en main RStudio : ses panneaux, les scripts, les projets et les notebooks.
- Connaître les types de données et les structures de données de R.
- Utiliser les opérateurs et accéder aux éléments d'un objet.
- Écrire des conditions, des boucles et ses propres fonctions.
- Installer et charger des packages ; trouver de l'aide.

2.1 R : un peu d'histoire

- Années 1970 : création du langage **S** aux Bell Laboratories (John Chambers et collègues) pour manipuler des données, faire des statistiques et des graphiques.
- Années 1990 : **R** est créé à l'Université d'Auckland par Ross Ihaka et Robert Gentleman, inspiré de S et du langage Scheme. Il est distribué sous licence libre GPL et développé par la *R Core Team* et une large communauté.
- Aujourd'hui, le CRAN (*Comprehensive R Archive Network*) propose plus de 20 000 packages, et R est l'un des deux langages de référence de la science des données, avec Python.

R est un langage **interprété** : il n'y a pas d'étape de compilation, chaque instruction est exécutée dès qu'on la soumet.

2.2 Introduction à RStudio

2.2.1 R et RStudio : le moteur et le tableau de bord

R est le langage et le logiciel qui exécute les calculs. **RStudio** est un **environnement de développement intégré** (*IDE*) : une interface qui rend le travail avec R plus confortable (éditeur de code, gestion des fichiers, graphiques, aide…). On peut comparer R au moteur d'une voiture et RStudio à son tableau de bord : RStudio ne fonctionne pas sans R, mais R fonctionne sans RStudio.

Installation, dans cet ordre :

1. installer R depuis le CRAN : cran.r-project.org ;
2. installer RStudio Desktop (gratuit) depuis posit.co/download/rstudio-desktop.

En cas de problème d'installation, Posit Cloud permet d'utiliser RStudio dans un navigateur web, gratuitement dans la limite d'un quota mensuel.

2.2.2 Les quatre panneaux

Au lancement, RStudio affiche quatre panneaux. Si vous n'en voyez que trois, c'est qu'aucun fichier n'est ouvert : *File > New File > R Script* fait apparaître l'éditeur.

Panneau	Onglet	À quoi il sert
Éditeur	—	écrire, enregistrer et exécuter des scripts et des notebooks
Console	*Console*	exécuter du code ; voir les résultats, avertissements et erreurs
	Terminal	ligne de commande du système (Git, fichiers…)
Environnement	*Environment*	liste des objets en mémoire ; un clic sur un tableau l'ouvre avec `View()` ; bouton *Import Dataset*
	History	toutes les commandes exécutées
Divers	*Files*	explorateur de fichiers du projet
	Plots	graphiques produits ; bouton *Export* pour les enregistrer
	Packages	packages installés ; cocher une case équivaut à `library()`
	Help	pages d'aide, ouvertes par `?fonction` ou la touche *F1*
	Viewer	sorties web (tableaux interactifs, applications Shiny, notebooks)

La disposition se règle dans *Tools > Global Options > Pane Layout*, et l'apparence (thème sombre, taille de police) dans *Appearance*.

2.2.3 Console ou script ?

Dans la console, on tape une expression après l'invite `>` et on valide avec *Entrée* :

```
2 + 1
```

```
#> [1] 3
```

Les flèches *haut* et *bas* rappellent les commandes précédentes. Si l'invite devient `+`, R attend la fin d'une commande incomplète (une parenthèse non fermée, par exemple) : complétez-la ou appuyez sur *Échap*.

La console est pratique pour un test rapide, mais **rien n'y est conservé**. On écrit donc le code dans un **script** (*File > New File > R Script*, extension `.R`), qu'on enregistre et qu'on exécute :

Action	Windows / Linux	macOS
exécuter la ligne courante ou la sélection	*Ctrl + Entrée*	*Cmd + Entrée*
exécuter tout le script	*Ctrl + Maj + Entrée*	*Cmd + Maj + Entrée*
enregistrer	*Ctrl + S*	*Cmd + S*

Dans un script, tout ce qui suit `#` est un **commentaire**, ignoré par R :

```
# Analyse des ventes de la boutique - version du 16/10/2026
prix_ht <- 49.90
prix_ttc <- prix_ht * 1.20   # TVA à 20 %
```

2.2.4 Les projets RStudio

Un **projet** RStudio (*File > New Project > New Directory*) regroupe dans un dossier tout ce qui concerne une analyse : données, scripts, résultats. Ouvrir le fichier `.Rproj` :

- fixe le **répertoire de travail** sur le dossier du projet, ce qui permet d'utiliser des **chemins relatifs** (`"donnees/ventes.csv"`) ;
- rouvre les fichiers sur lesquels on travaillait ;
- sépare proprement les projets les uns des autres.

> **Un projet RStudio par projet d'analyse** (et donc un pour ce cours, un pour votre projet de groupe). C'est la condition pour que votre code fonctionne aussi sur l'ordinateur de vos camarades et de l'enseignant.

2.2.5 Les notebooks : R Markdown et Quarto

Un **notebook** mêle dans un même fichier du **texte** (en Markdown), du **code** et ses **résultats** (tableaux, graphiques). C'est le format idéal pour les comptes rendus de TP et le rapport de projet : le document se régénère entièrement à partir du code.

- **R Markdown** (`.Rmd`, *File > New File > R Markdown*) : le format historique ; ces notes sont écrites en R Markdown.
- **Quarto** (`.qmd`, *File > New File > Quarto Document*) : son successeur, utilisé dans les supports de TP du cours.

La syntaxe est presque identique. Le code est placé dans des **blocs** (*chunks*) :

````
## Panier moyen

```{r}
paniers <- c(42.5, 18.0, 73.9)
mean(paniers)
```
````

Action	Windows / Linux	macOS
insérer un bloc de code	*Ctrl + Alt + I*	*Cmd + Option + I*
exécuter le bloc courant	*Ctrl + Maj + Entrée*	*Cmd + Maj + Entrée*
produire le document (*Knit / Render*)	*Ctrl + Maj + K*	*Cmd + Maj + K*

2.2.6 L'onglet Environnement

L'onglet *Environment* liste les objets créés pendant la session. Pour un tableau de données, il affiche le nombre d'observations et de variables ; le triangle bleu montre sa structure, et un clic sur son nom l'ouvre dans un visualiseur de type tableur (équivalent de `View()`).

Le bouton *Import Dataset* ouvre un assistant d'import (CSV, Excel, SPSS…) qui **génère le code R** correspondant : copiez ce code dans votre script plutôt que de refaire l'import à la souris à chaque fois.

Le balai efface tous les objets. Pour repartir vraiment de zéro, préférez **redémarrer R** (*Session > Restart R*) : si votre script ne fonctionne plus après un redémarrage, c'est qu'il dépendait d'un objet créé à la main.

2.2.7 Écrire du code efficacement

- **Complétion automatique** : tapez le début d'un nom puis *Tab* ; RStudio propose les objets, les fonctions et leurs arguments.
- **Aide contextuelle** : curseur sur un nom de fonction, puis *F1*.
- **Diagnostics** : RStudio signale dans la marge les parenthèses non fermées et les erreurs de syntaxe.

Raccourci	Windows / Linux	macOS
`<-` (assignation)	*Alt + -*	*Option + -*
`|>` ou `%>%` (*pipe*)	*Ctrl + Maj + M*	*Cmd + Maj + M*
commenter / décommenter	*Ctrl + Maj + C*	*Cmd + Maj + C*
réindenter / reformater	*Ctrl + I* / *Ctrl + Maj + A*	*Cmd + I* / *Cmd + Maj + A*
insérer une section de script	*Ctrl + Maj + R*	*Cmd + Maj + R*
vider la console	*Ctrl + L*	*Ctrl + L*
redémarrer R	*Ctrl + Maj + F10*	*Cmd + Maj + 0*
liste de tous les raccourcis	*Alt + Maj + K*	*Option + Maj + K*

2.2.8 Réglages recommandés

Dans *Tools > Global Options* :

- *General > Basic* : décocher **Restore .RData into workspace at startup** et régler **Save workspace to .RData on exit** sur **Never**. Votre script doit suffire à tout recréer ; un espace de travail restauré masque les oublis.
- *Code > Editing* : cocher **Use native pipe operator, |>** pour que *Ctrl + Maj + M* insère le *pipe* natif.
- *Code > Saving* : encodage par défaut **UTF-8** (indispensable pour les accents).

Le menu *Addins* donne accès à des extensions de l'interface, comme `esquisse` pour construire des graphiques à la souris (chapitre 4).

2.3 Premiers pas : R comme calculatrice

```
2 + 1
```

```
#> [1] 3
```

```
10 / 3
```

```
#> [1] 3.333
```

```r
2^10
```

```
#> [1] 1024
```

```r
sqrt(16)       # racine carrée
```

```
#> [1] 4
```

```r
exp(1)         # e
```

```
#> [1] 2.718
```

```r
log(100, base = 10)
```

```
#> [1] 2
```

```r
round(19.99 * 1.2, 2)
```

```
#> [1] 23.99
```

2.4 Variables et assignation

Le résultat d'une expression est affiché, mais pas conservé. Pour le conserver, on l'**assigne** à une variable avec `<-` :

```r
x <- 2 + 1     # assignation : rien n'est affiché
x              # affichage du contenu
```

```
#> [1] 3
```

```r
y <- x^2
y
```

```
#> [1] 9
```

```r
(z <- x + y)   # des parenthèses autour forcent l'affichage
```

```
#> [1] 12
```

Le signe `=` fonctionne aussi, mais l'usage en R est de réserver `=` aux arguments des fonctions.

Règles de nommage : lettres, chiffres, `_` et `.` ; le nom ne commence pas par un chiffre et ne contient pas d'espace. R **distingue les majuscules et les minuscules** (`x` et `X` sont deux objets différents).

Convention du cours : tout en minuscules, mots séparés par `_` (*snake_case*), noms explicites : `taux_conversion`, `panier_moyen`.

```r
panier_moyen <- 54.3
Panier_moyen   # erreur : la casse compte
```

```
#> Error: object 'Panier_moyen' not found
```

Gestion des objets en mémoire :

```r
premier <- 3
second <- premier  # copie de la valeur
premier <- 4       # modifier premier ne modifie pas second
```

```
second
```

```
#> [1] 3
```

```
rm(premier)        # suppression
exists("premier")
```

```
#> [1] FALSE
```

2.5 Types de données

Tout objet R possède un **type** (ou mode), une **longueur** et un contenu. Les fonctions `typeof()`, `class()` et `length()` permettent de les connaître.

2.5.1 Les types de base

Type	Exemple	Test	Conversion
`double` (réel)	`3.14`, `2`	`is.double()`	`as.double()`
`integer` (entier)	`2L`, `1:3`	`is.integer()`	`as.integer()`
`character` (texte)	`"Paris"`	`is.character()`	`as.character()`
`logical` (booléen)	`TRUE`, `FALSE`	`is.logical()`	`as.logical()`
`complex`	`2+3i`	`is.complex()`	`as.complex()`

`double` et `integer` sont tous deux **numériques** (`is.numeric()` vaut `TRUE`).

```
typeof(2)          # un nombre écrit sans L est un réel
```

```
#> [1] "double"
```

```
typeof(2L)         # le suffixe L crée un entier
```

```
#> [1] "integer"
```

```
typeof("Bonjour")
```

```
#> [1] "character"
```

```
typeof(TRUE)
```

```
#> [1] "logical"
```

```
z <- 2 + 3i
c(Re(z), Im(z), Mod(z))
```

```
#> [1] 2.000 3.000 3.606
```

`TRUE` et `FALSE` valent 1 et 0 dans un calcul. C'est très utile pour compter, par exemple les visiteurs qui ont acheté :

```
a_achete <- c(FALSE, TRUE, FALSE, FALSE, TRUE, FALSE, FALSE, FALSE)
sum(a_achete)      # nombre d'acheteurs
```

```
#> [1] 2
```

```
mean(a_achete)      # taux de conversion
```

```
#> [1] 0.25
```

2.5.2 Conversions

```
as.numeric("3.5")
```

```
#> [1] 3.5
```

```
as.integer(3.9)        # troncature, pas arrondi
```

```
#> [1] 3
```

```
as.character(42)
```

```
#> [1] "42"
```

```
as.numeric("trois")    # conversion impossible : NA avec un avertissement
```

```
#> Warning: NAs introduced by coercion
#> [1] NA
```

2.5.3 Valeurs spéciales

Valeur	Signification	Test
`NA`	valeur manquante (*Not Available*)	`is.na()`
`NULL`	objet vide, de longueur 0	`is.null()`
`NaN`	résultat indéfini (*Not a Number*), ex. `0/0`	`is.nan()`
`Inf`, `-Inf`	infini, ex. `1/0`	`is.infinite()`

```
ages <- c(21, NA, 23)
mean(ages)                  # NA contamine le calcul
```

```
#> [1] NA
```

```
mean(ages, na.rm = TRUE)    # on ignore les valeurs manquantes
```

```
#> [1] 22
```

```
length(NULL)
```

```
#> [1] 0
```

```
0 / 0
```

```
#> [1] NaN
```

> On ne teste **jamais** une valeur manquante avec `x == NA` (le résultat est toujours `NA`). On utilise `is.na(x)`.

2.6 Structures de données

La fonction `class()` indique la structure d'un objet. Les principales structures sont résumées ci-dessous.

Structure	Dimensions	Types des éléments
vecteur	1	un seul type
facteur	1	modalités d'une variable qualitative
matrice	2	un seul type
tableau (*array*)	n	un seul type
liste	1	types quelconques
data frame	2	un type par colonne

2.6.1 Vecteurs

Le vecteur est l'objet de base de R : un nombre seul est un vecteur de longueur 1. On le crée avec `c()` (*combine*).

```
ventes <- c(12500, 14000, 9800, 16200)
length(ventes)
```

```
#> [1] 4
```

```
names(ventes) <- c("Paris", "Lyon", "Lille", "Bordeaux")
ventes
```

```
#>    Paris     Lyon    Lille Bordeaux
#>    12500    14000     9800    16200
```

Tous les éléments d'un vecteur ont **le même type**. En cas de mélange, R convertit vers le type le plus général (logique → entier → réel → texte) :

```
c("deux", 1, TRUE)
```

```
#> [1] "deux" "1"    "TRUE"
```

```
c(0, FALSE)
```

```
#> [1] 0 0
```

2.6.2 Facteurs

Un **facteur** représente une variable **qualitative** (catégorielle). Il stocke les valeurs possibles, appelées **niveaux** (*levels*).

```
canal <- factor(c("Email", "Réseaux sociaux", "Email", "Direct", "Réseaux
 ↪  sociaux"))
canal
```

```
#> [1] Email           Réseaux sociaux Email           Direct
#> [5] Réseaux sociaux
#> Levels: Direct Email Réseaux sociaux
```

```
levels(canal)                          # ordre alphabétique par défaut
```

```
#> [1] "Direct"          "Email"           "Réseaux sociaux"
```

```r
table(canal)                              # effectifs par modalité
```

```
#> canal
#>          Direct           Email Réseaux sociaux
#>               1               2               2
```

```r
canal <- relevel(canal, ref = "Direct")   # changer la modalité de référence
levels(canal)
```

```
#> [1] "Direct"          "Email"           "Réseaux sociaux"
```

Pour une variable **ordinale** (avec un ordre naturel), on utilise un facteur ordonné :

```r
satisfaction <- factor(c("Satisfait", "Très satisfait", "Peu satisfait",
 ↪  "Satisfait"),
                       levels = c("Peu satisfait", "Satisfait", "Très
                        ↪  satisfait"),
                       ordered = TRUE)
satisfaction
```

```
#> [1] Satisfait      Très satisfait Peu satisfait  Satisfait
#> Levels: Peu satisfait < Satisfait < Très satisfait
```

```r
satisfaction >= "Satisfait"
```

```
#> [1]  TRUE  TRUE FALSE  TRUE
```

2.6.3 Matrices et tableaux

Une matrice est un vecteur avec deux dimensions. Elle se remplit **colonne par colonne** par défaut.

```r
m <- matrix(1:6, nrow = 3, ncol = 2)
m
```

```
#>      [,1] [,2]
#> [1,]    1    4
#> [2,]    2    5
#> [3,]    3    6
```

```r
matrix(1:6, nrow = 3, byrow = TRUE)   # remplissage ligne par ligne
```

```
#>      [,1] [,2]
#> [1,]    1    2
#> [2,]    3    4
#> [3,]    5    6
```

```r
dim(m)
```

```
#> [1] 3 2
```

```r
t(m)                                  # transposée
```

```
#>      [,1] [,2] [,3]
#> [1,]    1    2    3
#> [2,]    4    5    6
```

```r
a <- array(1:24, dim = c(2, 4, 3))     # tableau à 3 dimensions
dim(a)
```

```
#> [1] 2 4 3
```

Opérations matricielles utiles en analyse de données (l'ACP et la régression reposent sur elles) :

Opération	Syntaxe
produit matriciel	`A %*% B`
produit terme à terme	`A * B`
transposée, inverse	`t(A)`, `solve(A)`
$A^\top B$	`crossprod(A, B)`
déterminant, diagonale	`det(A)`, `diag(A)`
juxtaposer, superposer	`cbind()`, `rbind()`

```r
A <- matrix(c(1, 3, 2, 2, 2, 1, 3, 1, 3), ncol = 3)
round(A %*% solve(A), 10)       # matrice identité
```

```
#>      [,1] [,2] [,3]
#> [1,]    1    0    0
#> [2,]    0    1    0
#> [3,]    0    0    1
```

`%/%` est la division **entière**, appliquée terme à terme ; ce n'est pas une division matricielle. Pour « diviser » par une matrice, on multiplie par son inverse : `B %*% solve(A)`.

2.6.4 Listes

Une liste peut contenir des éléments de **types et de tailles différents**, y compris d'autres listes. Beaucoup de fonctions R renvoient leurs résultats sous forme de liste (par exemple `lm()`).

```r
client <- list(id = "C0042", paniers = c(54.9, 23.5, 89.0), newsletter = TRUE)
client
```

```
#> $id
#> [1] "C0042"
#>
#> $paniers
#> [1] 54.9 23.5 89.0
#>
#> $newsletter
#> [1] TRUE
```

```r
str(client)       # structure compacte, très utile
```

```
#> List of 3
#>  $ id        : chr "C0042"
#>  $ paniers   : num [1:3] 54.9 23.5 89
#>  $ newsletter: logi TRUE
```

2.6.5 Data frames

Le **data frame** est la structure centrale de l'analyse de données : un tableau dont chaque **colonne est une variable** (un vecteur, avec son propre type) et chaque **ligne une observation**. Techniquement, c'est une liste de vecteurs de même longueur.

```r
commandes <- data.frame(
  client  = c("C01", "C02", "C01", "C03", "C02"),
  montant = c(54.9, 23.5, 89.0, 42.0, 17.9),
  promo   = c(FALSE, TRUE, FALSE, FALSE, TRUE)
)
commandes
```

```
#>   client montant promo
#> 1    C01    54.9 FALSE
#> 2    C02    23.5  TRUE
#> 3    C01    89.0 FALSE
#> 4    C03    42.0 FALSE
#> 5    C02    17.9  TRUE
```

```r
dim(commandes)
```

```
#> [1] 5 3
```

```r
str(commandes)
```

```
#> 'data.frame':    5 obs. of  3 variables:
#>  $ client : chr  "C01" "C02" "C01" "C03" ...
#>  $ montant: num  54.9 23.5 89 42 17.9
#>  $ promo  : logi  FALSE TRUE FALSE FALSE TRUE
```

```r
summary(commandes)
```

```
#>     client             montant       promo
#>  Length:5           Min.   :17.9   Mode :logical
#>  Class :character   1st Qu.:23.5   FALSE:3
#>  Mode  :character   Median :42.0   TRUE :2
#>                     Mean   :45.5
#>                     3rd Qu.:54.9
#>                     Max.   :89.0
```

Le *tidyverse* (chapitre 3) utilise une variante moderne, le **tibble**, qui s'affiche de façon plus lisible et se comporte de façon plus prévisible.

2.7 Opérateurs

2.7.1 Opérateurs arithmétiques

`+`, `-`, `*`, `/`, `^` (puissance), `%%` (reste de la division entière, ou modulo), `%/%` (quotient de la division entière).

Ils sont **vectorisés** : l'opération s'applique élément par élément, sans boucle.

```r
prix <- c(19.9, 49.0, 5.5)
quantite <- c(2, 1, 10)
prix * quantite           # chiffre d'affaires par produit
```

```
#> [1] 39.8 49.0 55.0
```

```r
sum(prix * quantite)
```

```
#> [1] 143.8
```

```r
prix * 1.2                # le scalaire est appliqué à chaque élément
```

```
#> [1] 23.88 58.80  6.60
```

> **Recyclage** : si deux vecteurs n'ont pas la même longueur, R répète le plus court. C'est pratique avec un scalaire, mais source d'erreurs silencieuses sinon. R n'affiche un avertissement que si la longueur du plus long n'est pas un multiple de celle du plus court.
>
> ```r
> c(1, 2, 3, 4) + c(10, 20)      # aucun avertissement !
> ```
>
> ```
> #> [1] 11 22 13 24
> ```

2.7.2 Opérateurs de comparaison

`<`, `<=`, `>`, `>=`, `==` (égal), `!=` (différent). Le résultat est un vecteur logique.

```r
paniers <- c(42.5, 18.0, 73.9, 120.0)
paniers > 50
```

```
#> [1] FALSE FALSE  TRUE  TRUE
```

```r
paniers == 18
```

```
#> [1] FALSE  TRUE FALSE FALSE
```

> Les nombres réels sont stockés de façon approchée. Ne comparez pas deux réels avec `==` ; utilisez `all.equal()` ou une tolérance.
>
> ```r
> 0.9 == 1.1 - 0.2
> ```
>
> ```
> #> [1] FALSE
> ```
>
> ```r
> isTRUE(all.equal(0.9, 1.1 - 0.2))
> ```
>
> ```
> #> [1] TRUE
> ```

2.7.3 Opérateurs logiques

Opérateur	Sens	Usage
`!`	NON	vectorisé
`&`, `|`	ET, OU	**vectorisés** : filtres, sélections
`&&`, `||`	ET, OU	**un seul** `TRUE`/`FALSE` : conditions de `if`
`xor()`	OU exclusif	vectorisé
`any()`, `all()`	au moins un / tous `TRUE`	résumé d'un vecteur

```r
a <- c(TRUE, TRUE, FALSE, FALSE)
b <- c(TRUE, FALSE, TRUE, FALSE)
a & b
```

```
#> [1]  TRUE FALSE FALSE FALSE
```

```
a | b
```

```
#> [1]  TRUE  TRUE  TRUE FALSE
```

```
any(a); all(a)
```

```
#> [1] TRUE
```

```
#> [1] FALSE
```

```
a && b      # erreur depuis R 4.3 : && n'accepte que des valeurs de longueur 1
```

```
#> Error in a && b: 'length = 4' in coercion to 'logical(1)'
```

`&&` et `||` n'évaluent leur second argument que si c'est nécessaire, ce qui permet d'écrire `!is.null(x) && x > 0` sans erreur quand `x` vaut `NULL`.

L'opérateur `%in%` teste l'appartenance à un ensemble :

```
c("Lyon", "Nice") %in% c("Paris", "Lyon", "Lille")
```

```
#> [1]  TRUE FALSE
```

2.8 Accéder aux éléments et les modifier

Trois opérateurs d'accès : `[ ]`, `[[ ]]` et `$`.

2.8.1 Vecteurs

On peut indexer par **position**, par **position négative** (exclusion), par **vecteur logique** ou par **nom**.

```
x <- c(4, 7, 3, 5, 0)
x[2]              # deuxième élément (les indices commencent à 1)
```

```
#> [1] 7
```

```
x[c(1, 3)]
```

```
#> [1] 4 3
```

```
x[-2]             # tout sauf le deuxième
```

```
#> [1] 4 3 5 0
```

```
x[x > 3]          # filtrage logique
```

```
#> [1] 4 7 5
```

```
which(x > 3)      # positions des éléments TRUE
```

```
#> [1] 1 2 4
```

```
which.max(x)      # position du maximum
```

```
#> [1] 2
```

```
x[x == 0] <- 10   # remplacement
x
```

```
#> [1]  4  7  3  5 10
```

```
ventes["Lyon"]
```

```
#>  Lyon
#> 14000
```

2.8.2 Matrices et data frames : `[ligne, colonne]`

Laisser un indice vide signifie « tout ».

```
m <- matrix(1:9, nrow = 3)
m[1, 2]                 # ligne 1, colonne 2
```

```
#> [1] 4
```

```
m[, 2]                  # colonne 2 (renvoyée sous forme de vecteur)
```

```
#> [1] 4 5 6
```

```
m[, 2, drop = FALSE]    # reste une matrice
```

```
#>      [,1]
#> [1,]    4
#> [2,]    5
#> [3,]    6
```

```
commandes[2, ]                          # deuxième commande
```

```
#>   client montant promo
#> 2    C02    23.5  TRUE
```

```
commandes$montant                       # une colonne, sous forme de vecteur
```

```
#> [1] 54.9 23.5 89.0 42.0 17.9
```

```
commandes[commandes$montant > 40, ]   # filtrer des lignes
```

```
#>   client montant promo
#> 1    C01    54.9 FALSE
#> 3    C01    89.0 FALSE
#> 4    C03    42.0 FALSE
```

```
commandes$montant_ht <- round(commandes$montant / 1.2, 2)   # nouvelle colonne
commandes
```

```
#>   client montant promo montant_ht
#> 1    C01    54.9 FALSE      45.75
#> 2    C02    23.5  TRUE      19.58
#> 3    C01    89.0 FALSE      74.17
#> 4    C03    42.0 FALSE      35.00
#> 5    C02    17.9  TRUE      14.92
```

2.8.3 Listes : `[ ]` contre `[[ ]]`

- `liste[1]` renvoie une **liste** contenant le premier élément ;
- `liste[[1]]` ou `liste$nom` renvoie **le contenu** de l'élément.

Image utile : si la liste est un train, `[ ]` renvoie un train plus court, `[[ ]]` renvoie le contenu d'un wagon.

```r
client[2]           # une liste
```

```
#> $paniers
#> [1] 54.9 23.5 89.0
```

```r
client[[2]]         # un vecteur
```

```
#> [1] 54.9 23.5 89.0
```

```r
client$paniers[3]   # troisième panier
```

```
#> [1] 89
```

```r
client$ville <- "Paris"    # ajout d'un élément
names(client)
```

```
#> [1] "id"         "paniers"    "newsletter" "ville"
```

2.9 Structures algorithmiques

2.9.1 Branchements conditionnels : `if`, `else`

```r
panier <- 65
if (panier >= 100) {
  print("Livraison gratuite")
} else if (panier >= 50) {
  print("Livraison à 2,90 €")
} else {
  print("Livraison à 4,90 €")
}
```

```
#> [1] "Livraison à 2,90 €"
```

La condition d'un `if` doit être **un seul** `TRUE` ou `FALSE`. Pour appliquer une condition à **chaque élément** d'un vecteur, on utilise la fonction vectorisée `ifelse()` (ou `dplyr::case_when()` au chapitre suivant) :

```r
paniers <- c(42.5, 18.0, 73.9, 120.0)
ifelse(paniers >= 50, "Gros panier", "Petit panier")
```

```
#> [1] "Petit panier" "Petit panier" "Gros panier"  "Gros panier"
```

`switch()` choisit une branche selon la valeur d'un texte :

```r
canal <- "Email"
switch(canal,
       "Email" = 0.05,            # coût par contact
       "Réseaux sociaux" = 0.80,
       "Courrier" = 1.20)
```

```
#> [1] 0.05
```

2.9.2 Boucles `for`

Une boucle `for` répète un bloc pour chaque élément d'une séquence.

```r
for (ville in c("Paris", "Lyon", "Lille")) {
  print(paste("Rapport des ventes :", ville))
}
```

```
#> [1] "Rapport des ventes : Paris"
#> [1] "Rapport des ventes : Lyon"
#> [1] "Rapport des ventes : Lille"
```

Pour parcourir les positions d'un vecteur, utilisez `seq_along()` (plutôt que `1:length(x)`, qui donne `1 0` quand `x` est vide) et **préallouez** le résultat :

```r
paniers <- c(42.5, 18.0, 73.9, 120.0)
frais_port <- numeric(length(paniers))      # préallocation
for (i in seq_along(paniers)) {
  frais_port[i] <- if (paniers[i] >= 50) 0 else 4.90
}
frais_port
```

```
#> [1] 4.9 4.9 0.0 0.0
```

2.9.3 Boucles `while` et `repeat`

`while` répète tant qu'une condition est vraie ; `break` sort d'une boucle, `next` passe directement à l'itération suivante.

```r
# Combien de mois pour atteindre 10 000 abonnés avec une croissance de 8 % par
↪  mois ?
abonnes <- 2500
mois <- 0
while (abonnes < 10000) {
  abonnes <- abonnes * 1.08
  mois <- mois + 1
}
mois
```

```
#> [1] 19
```

2.9.4 Boucle ou vectorisation ?

En R, la plupart des opérations sont déjà vectorisées. Un code vectorisé est plus court, plus lisible et beaucoup plus rapide.

```r
# Avec une boucle
frais_port <- numeric(length(paniers))
for (i in seq_along(paniers)) frais_port[i] <- if (paniers[i] >= 50) 0 else
↪  4.90
# Vectorisé : préférable
frais_port <- ifelse(paniers >= 50, 0, 4.90)
frais_port
```

```
#> [1] 4.9 4.9 0.0 0.0
```

Gardez les boucles pour les cas où chaque itération dépend de la précédente (comme l'exemple des abonnés) ou pour répéter une tâche complexe (lire plusieurs fichiers, simuler plusieurs scénarios).

2.10 Fonctions et programmation modulaire

Un programme lisible est découpé en **petites unités** qui font chacune une seule chose, ont un nom explicite et peuvent être testées séparément.

2.10.1 Écrire une fonction

```
prix_ttc <- function(prix_ht, tva = 0.20) {
  round(prix_ht * (1 + tva), 2)
}
prix_ttc(100)
```

```
#> [1] 120
```

```
prix_ttc(100, tva = 0.055)          # taux réduit
```

```
#> [1] 105.5
```

```
prix_ttc(c(10, 25, 49.9))           # vectorisée « gratuitement »
```

```
#> [1] 12.00 30.00 59.88
```

Anatomie d'une fonction :

- un **nom** (un verbe ou un nom explicite) ;
- des **arguments**, éventuellement avec des **valeurs par défaut** ;
- un **corps** ; la **dernière expression évaluée est renvoyée**. `return()` sert surtout à sortir plus tôt.

```
resumer <- function(x, arrondi = 2) {
  if (!is.numeric(x)) {
    stop("x doit être numérique")       # message d'erreur explicite
  }
  c(moyenne    = round(mean(x, na.rm = TRUE), arrondi),
    mediane    = round(median(x, na.rm = TRUE), arrondi),
    n_manquant = sum(is.na(x)))
}
resumer(c(42.5, 18.0, NA, 73.9, 120.0))
```

```
#>    moyenne    mediane n_manquant
#>       63.6       58.2        1.0
```

Portée des variables : les variables créées dans une fonction sont **locales** ; elles n'existent plus après l'appel et ne modifient pas les variables du même nom à l'extérieur. Une fonction doit recevoir ce dont elle a besoin **par ses arguments** et transmettre son résultat **par sa valeur de retour**.

2.10.2 Fonctions et procédures

On distingue parfois :

- les **fonctions** (au sens strict), qui calculent et **renvoient** une valeur sans rien modifier d'autre : `prix_ttc()`, `resumer()` ;
- les **procédures**, appelées pour leurs **effets de bord** : afficher, écrire un fichier, tracer un graphique. Elles renvoient souvent leur entrée de façon invisible avec `invisible()`, ce qui permet de les insérer dans une chaîne de *pipes*.

```r
decrire_table <- function(df) {
  cat("Lignes :", nrow(df), "| Colonnes :", ncol(df), "\n")
  cat("Valeurs manquantes :", sum(is.na(df)), "\n")
  invisible(df)
}
decrire_table(commandes)
```

```
#> Lignes : 5 | Colonnes : 4
#> Valeurs manquantes : 0
```

2.10.3 Appliquer une fonction à chaque élément

Les fonctions de la famille `apply` évitent d'écrire des boucles :

```r
paniers_par_canal <- list(email = c(54, 61, 48), social = c(32, 29, 41, 25),
                          direct = c(88, 95))
sapply(paniers_par_canal, mean)        # simplifie le résultat en vecteur
```

```
#>  email social direct
#>  54.33  31.75  91.50
```

```r
lapply(paniers_par_canal, range)       # renvoie toujours une liste
```

```
#> $email
#> [1] 48 61
#>
#> $social
#> [1] 25 41
#>
#> $direct
#> [1] 88 95
```

```r
apply(matrix(1:6, nrow = 2), 2, sum)   # sur les colonnes (1 = lignes)
```

```
#> [1]  3  7 11
```

2.10.4 Découper un programme en modules

Dès qu'un projet grandit, on range les fonctions dans des fichiers séparés, chargés avec `source()`. Une organisation simple :

```
mon_projet/
|-- mon_projet.Rproj
|-- donnees/
|   |-- brutes/           # données d'origine, jamais modifiées
|   `-- propres/          # données nettoyées par le code
|-- R/
|   |-- nettoyage.R       # fonctions de préparation des données
|   |-- indicateurs.R     # calculs (RFM, taux de conversion...)
|   `-- graphiques.R      # fonctions de visualisation
|-- analyse.qmd           # notebook principal : appelle les fonctions
`-- resultats/            # figures et tableaux produits
```

```r
# En tête de analyse.qmd
source("R/nettoyage.R")
source("R/indicateurs.R")
```

Règles d'un code modulaire

- Ne jamais copier-coller un bloc plus de deux fois : en faire une fonction.
- Une fonction = une tâche, un nom explicite, des arguments, une valeur de retour.
- Pas de variables globales cachées dans les fonctions.
- Les données brutes ne sont jamais modifiées à la main : tout passe par le code.
- Commenter le **pourquoi**, pas le **comment**.

2.11 Packages et aide

2.11.1 Installer et charger un package

Un **package** (ou extension) regroupe des fonctions, de la documentation et parfois des jeux de données. Certains sont chargés au démarrage (`base`, `stats`, `graphics`, `utils`, `datasets`…). Pour les autres :

```
install.packages("questionr")   # une seule fois par ordinateur (télécharge
↪  depuis le CRAN)
library(questionr)              # à chaque session, en début de script
```

Dans RStudio, l'onglet *Packages* permet aussi d'installer (bouton *Install*) et de mettre à jour (bouton *Update*) les packages.

On peut appeler une fonction sans charger tout le package, avec `package::fonction()`, ce qui rend explicite la provenance de la fonction :

```
stats::median(c(3, 1, 2))
```

```
#> [1] 2
```

`sessionInfo()` affiche la version de R et des packages chargés : à joindre à toute demande d'aide et à votre rapport de projet.

2.11.2 Trouver de l'aide

```
?mean                    # page d'aide d'une fonction (équivalent :
↪  help("mean"), ou F1)
??regression             # recherche par mot-clé (équivalent : help.search())
example(mean)            # exécute les exemples de la page d'aide
vignette("dplyr")        # documentation longue d'un package
```

Une page d'aide se lit dans cet ordre : *Description*, *Usage* (les arguments et leurs valeurs par défaut), *Arguments*, *Value* (ce que renvoie la fonction), puis *Examples*, à exécuter.

Autres ressources : les *cheatsheets* de Posit (*Help* > *Cheat Sheets* dans RStudio), Stack Overflow, et les messages d'erreur eux-mêmes, à lire attentivement.

2.12 Résumé

- **RStudio** : éditeur, console, environnement, fichiers/graphiques/aide. Écrire dans des **scripts** ou des **notebooks**, travailler dans un **projet**, ne jamais restaurer l'espace de travail.
- `<-` pour assigner ; types `double`, `integer`, `character`, `logical` ; structures vecteur, facteur, matrice, liste, data frame.

- Accès : `[ ]`, `[[ ]]`, `$` ; filtres logiques avec `&`, `|`, `%in%`.
- `if` / `else` pour **une** condition ; `ifelse()` pour un vecteur ; `for`, `while` pour répéter ; préférer la vectorisation.
- Fonctions : un nom, des arguments, une valeur de retour ; modules rangés dans `R/` et chargés avec `source()`.
- `install.packages()` une fois, `library()` à chaque session ; `?fonction` pour l'aide.

2.13 Exercices

Exercice 2.1 — Prise en main de RStudio.

1. Créer un projet RStudio `data-science-R` et y créer un dossier `donnees`.
2. Appliquer les réglages recommandés (espace de travail, *pipe* natif).
3. Créer un script `premier-script.R` qui calcule le prix TTC de trois articles et l'exécuter ligne par ligne, puis en entier.
4. Créer un document Quarto, y insérer un bloc de code avec le raccourci clavier, puis produire le document HTML.
5. Redémarrer R et vérifier que le script s'exécute toujours.

Exercice 2.2 — Types et structures.

1. Que permet une liste par rapport à un vecteur ?
2. Que renvoient `c("deux", 1, TRUE)`, `c(0, FALSE)` et `c(0, "FALSE")` ? Pourquoi ?
3. Créer un facteur de longueur 5 dont les modalités sont `"Google"`, `"Instagram"` et `"TikTok"`, chacune présente au moins une fois, avec `"Google"` comme modalité de référence.
4. Créer la matrice $\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$, puis la transformer en data frame.

Solution

```
# 1. Une liste peut contenir des éléments de types et de longueurs
 ↪  différents.
# 2. Conversion vers le type le plus général :
c("deux", 1, TRUE)   # tout devient du texte
```

```
#> [1] "deux" "1"    "TRUE"
```

```
c(0, FALSE)          # FALSE devient 0
```

```
#> [1] 0 0
```

```
c(0, "FALSE")        # tout devient du texte
```

```
#> [1] "0"     "FALSE"
```

```
# 3.
source_trafic <- factor(c("TikTok", "Google", "Instagram", "Google",
 ↪  "TikTok"))
source_trafic <- relevel(source_trafic, ref = "Google")
levels(source_trafic)
```

```
#> [1] "Google"    "Instagram" "TikTok"
```

```r
# 4.
m <- matrix(1:4, nrow = 2)
as.data.frame(m)
```

```
#>   V1 V2
#> 1  1  3
#> 2  2  4
```

Exercice 2.3 — Vecteurs et indexation. Une campagne a généré les nombres de clics journaliers `clics <- c(120, 340, 95, 410, 220, 180, 60)` (du lundi au dimanche).

1. Nommer les éléments avec les jours de la semaine.
2. Calculer la part de chaque jour dans le total des clics.
3. Extraire les jours avec plus de 200 clics.
4. Quel jour a eu le plus de clics ? le moins ?
5. Normaliser les clics entre 0 et 1 : $(x - \min x)/(\max x - \min x)$.

Solution

```r
clics <- c(120, 340, 95, 410, 220, 180, 60)
names(clics) <- c("lun", "mar", "mer", "jeu", "ven", "sam", "dim")
round(clics / sum(clics), 3)
```

```
#>   lun   mar   mer   jeu   ven   sam   dim
#> 0.084 0.239 0.067 0.288 0.154 0.126 0.042
```

```r
clics[clics > 200]
```

```
#> mar jeu ven
#> 340 410 220
```

```r
names(which.max(clics)); names(which.min(clics))
```

```
#> [1] "jeu"
#> [1] "dim"
```

```r
(clics - min(clics)) / (max(clics) - min(clics))
```

```
#>    lun    mar    mer    jeu    ven    sam    dim
#> 0.1714 0.8000 0.1000 1.0000 0.4571 0.3429 0.0000
```

Exercice 2.4 — Accès aux éléments.

1. Extraire le troisième élément du vecteur `x <- c(3, 5, 6, 9)`.
2. Extraire les éléments de `x` supérieurs à 5.
3. Extraire les éléments de `x` qui ne sont pas dans l'intervalle $[4, 6]$.
4. Déterminer le maximum de `x` et donner sa position.
5. Remplacer ce maximum par le minimum.
6. Remplacer par `NA` les éléments négatifs de la matrice $M = \begin{pmatrix} 1 & -2 & 3 \\ -4 & 5 & -6 \end{pmatrix}$, puis extraire la deuxième colonne de la matrice obtenue.
7. Soit `L <- list(nombre = rnorm(10), lettre = LETTERS[1:26], mois = month.abb)`. Extraire le troisième élément de `L`, d'abord par sa position, puis par son nom ; extraire tous les éléments de `L` sauf le deuxième.
8. Toujours par le nom, ajouter 10 aux valeurs du premier élément de `L` (le résultat doit modifier `L`).

9. Soit `df <- data.frame(x = 1:10, y = letters[1:10], z = rev(letters[1:10]))`. Afficher les valeurs de la variable `y`, puis uniquement les colonnes `x` et `z`.

Solution

```
x <- c(3, 5, 6, 9)
x[3]
```

```
#> [1] 6
```

```
x[x > 5]
```

```
#> [1] 6 9
```

```
x[x < 4 | x > 6]
```

```
#> [1] 3 9
```

```
max(x); which.max(x)
```

```
#> [1] 9
#> [1] 4
```

```
x[which.max(x)] <- min(x)
x
```

```
#> [1] 3 5 6 3
```

```
M <- matrix(c(1, -4, -2, 5, 3, -6), nrow = 2)
M[M < 0] <- NA
M
```

```
#>      [,1] [,2] [,3]
#> [1,]    1   NA    3
#> [2,]   NA    5   NA
```

```
M[, 2]
```

```
#> [1] NA  5
```

```
set.seed(1)
L <- list(nombre = rnorm(10), lettre = LETTERS[1:26], mois = month.abb)
L[[3]]
```

```
#>  [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov"
#> [12] "Dec"
```

```
L$mois
```

```
#>  [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov"
#> [12] "Dec"
```

```
L[-2]
```

```
#> $nombre
#>  [1] -0.6265  0.1836 -0.8356  1.5953  0.3295 -0.8205  0.4874  0.7383
#>  [9]  0.5758 -0.3054
#>
#> $mois
#>  [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov"
#> [12] "Dec"
```

```r
L$nombre <- L$nombre + 10

df <- data.frame(x = 1:10, y = letters[1:10], z = rev(letters[1:10]))
df$y
```

```
#>  [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j"
```

```r
df[, c("x", "z")]
```

```
#>     x z
#> 1   1 j
#> 2   2 i
#> 3   3 h
#> 4   4 g
#> 5   5 f
#> 6   6 e
#> 7   7 d
#> 8   8 c
#> 9   9 b
#> 10 10 a
```

Exercice 2.5 — Conditions et boucles.

1. Écrire une boucle qui affiche les nombres de 1 à 20 en remplaçant les multiples de 3 par `"Fizz"`, ceux de 5 par `"Buzz"` et ceux de 15 par `"FizzBuzz"`.
2. Réécrire la même chose **sans boucle**, à l'aide de `ifelse()`.

Solution

```r
for (i in 1:20) {
  if (i %% 15 == 0) {
    print("FizzBuzz")
  } else if (i %% 3 == 0) {
    print("Fizz")
  } else if (i %% 5 == 0) {
    print("Buzz")
  } else {
    print(i)
  }
}
```

```
#> [1] 1
#> [1] 2
#> [1] "Fizz"
#> [1] 4
#> [1] "Buzz"
#> [1] "Fizz"
#> [1] 7
#> [1] 8
#> [1] "Fizz"
#> [1] "Buzz"
#> [1] 11
#> [1] "Fizz"
#> [1] 13
#> [1] 14
#> [1] "FizzBuzz"
```

```
#> [1] 16
#> [1] 17
#> [1] "Fizz"
#> [1] 19
#> [1] "Buzz"
```

```
i <- 1:20
ifelse(i %% 15 == 0, "FizzBuzz",
       ifelse(i %% 3 == 0, "Fizz",
              ifelse(i %% 5 == 0, "Buzz", i)))
```

```
#>  [1] "1"        "2"        "Fizz"     "4"        "Buzz"     "Fizz"
#>  [7] "7"        "8"        "Fizz"     "Buzz"     "11"       "Fizz"
#> [13] "13"       "14"       "FizzBuzz" "16"       "17"       "Fizz"
#> [19] "19"       "Buzz"
```

Exercice 2.6 — Fonctions.

1. Écrire une fonction `taux_conversion(visiteurs, acheteurs)` qui renvoie le taux de conversion en pourcentage, arrondi à une décimale, et qui produit une erreur explicite si `acheteurs > visiteurs`.
2. L'appliquer aux campagnes : visiteurs `c(1200, 800, 1500)`, acheteurs `c(36, 40, 30)`.

Solution

```
taux_conversion <- function(visiteurs, acheteurs) {
  if (any(acheteurs > visiteurs)) {
    stop("Il ne peut pas y avoir plus d'acheteurs que de visiteurs")
  }
  round(100 * acheteurs / visiteurs, 1)
}
taux_conversion(c(1200, 800, 1500), c(36, 40, 30))
```

```
#> [1] 3 5 2
```

Exercice 2.7 — Moindres carrés « à la main ». On considère le modèle $\text{mpg} = \beta_0 + \beta_1\,\text{hp} + \beta_2\,\text{wt} + \varepsilon$ sur le jeu `mtcars`. L'estimateur des moindres carrés ordinaires est $\hat{\beta} = (X^\top X)^{-1} X^\top y$.

1. Créer le vecteur `y` (consommation `mpg`) et la matrice `X` à trois colonnes : une colonne de 1, `hp` et `wt`.
2. Calculer $\hat{\beta}$ par calcul matriciel et comparer avec `lm(mpg ~ hp + wt, data = mtcars)`.

Solution

```
y <- mtcars$mpg
X <- cbind(constante = 1, hp = mtcars$hp, wt = mtcars$wt)
solve(t(X) %*% X) %*% t(X) %*% y
```

```
#>                [,1]
#> constante 37.22727
#> hp        -0.03177
#> wt        -3.87783
```

```r
coef(lm(mpg ~ hp + wt, data = mtcars))
```

```
#> (Intercept)          hp          wt
#>    37.22727    -0.03177    -3.87783
```

Pour aller plus loin

- J. Barnier, *Introduction à R et au tidyverse*, chapitres « Prise en main » et « Premier travail avec des données » (juba.github.io/tidyverse).
- H. Wickham et al., *R for Data Science*, 2e éd., chapitres « Workflow » (r4ds.hadley.nz).
- Documentation de RStudio : docs.posit.co/ide/user.